N E U R O L A B

Interactive Neural Network Learning Laboratory

Project Documentation
Prepared for Scilab GUI Hackathon — Final Submission

Built with Scilab 2026.1.0

Application	NEURO LAB
Platform	Scilab (Windows / Linux / macOS)
Category	AI / Machine Learning — Educational Tool
Datasets	XOR · AND · OR

Table of Contents

1	Project Title
2	Project Description
3	Problem Statement
4	Objective
5	Intended Users
6	Software Requirements
7	Toolboxes Used
8	Project Structure
9	Datasets
10	Neural Network Architecture
11	Neural Network Training
12	Activation Function
13	Loss Function
14	Backpropagation
15	GUI Features
16	Training Loss Graph
17	Neural Network Visualization
18	Prediction Table
19	Accuracy
20	Network Inspection
21	Mathematics Feature
22	Reset Feature
23	Steps to Run the Application
24	Sample Result
25	Expected Output
26	Advantages
27	Limitations
28	Future Enhancements
29	References
30	Conclusion

1. Project Title

NEURO LAB — Interactive Neural Network Learning Laboratory

2. Project Description

NEURO LAB is an interactive educational application developed using **Scilab** to demonstrate the basic working of an Artificial Neural Network.

The application allows users to select simple logical datasets such as:

- XOR
- AND
- OR

Users can modify important neural-network parameters such as:

- Learning Rate
- Number of Epochs
- Number of Hidden Neurons

After training, the application displays:

- Training accuracy
- Final loss
- Prediction results
- Prediction probabilities
- Training loss graph
- Neural network architecture
- Network weights
- Learning insights

The main purpose of the project is to make neural-network concepts easier to understand through **visual and interactive learning instead of only mathematical theory**.

The application implements forward propagation, sigmoid activation, binary cross-entropy loss and gradient-descent-based backpropagation.

3. Problem Statement

Neural networks are an important concept in Artificial Intelligence and Machine Learning, but beginners often find it difficult to understand how data moves through neurons, how weights are updated, and how a network learns from errors.

Traditional learning methods mainly depend on mathematical equations, static diagrams and theoretical explanations. This makes it difficult for students to visualize the actual learning process.

Therefore, there is a need for a simple interactive educational application that allows students to experiment with neural networks and immediately observe the effect of changing parameters.

Solution

NEURO LAB addresses this problem by providing a visual neural-network learning environment where users can train a small neural network and observe its predictions, accuracy, loss and network structure.

4. Objective

The main objectives of NEURO LAB are:

1. To demonstrate the basic working of an Artificial Neural Network.
2. To provide an interactive environment for neural-network experimentation.
3. To demonstrate forward propagation.
4. To demonstrate backpropagation and gradient descent.
5. To visualize the neural-network architecture.
6. To display training loss graphically.
7. To show actual and predicted outputs.
8. To allow users to change learning parameters.
9. To help students understand how hidden neurons affect learning.
10. To provide a simple educational tool for understanding neural-network fundamentals.

5. Intended Users

The application is mainly intended for:

- AI/ML students
- Computer Science students
- Beginners learning Neural Networks
- Students studying Machine Learning
- Faculty demonstrating neural-network concepts
- Anyone interested in understanding basic neural-network training

6. Software Requirements

Operating System

Windows / Linux / macOS

Software

Scilab 2026.1.0 or compatible Scilab version

Programming Language

Scilab

Hardware

Basic computer/laptop capable of running Scilab. No GPU is required because the application uses a small educational dataset and a small neural network.

7. Toolboxes Used

The project primarily uses **Scilab's built-in functionality**. No external machine-learning library is required. The application uses Scilab functionality for:

- GUI creation
- Matrix operations
- Mathematical calculations
- Plotting
- Graph visualization
- Event callbacks

8. Project Structure

The project contains the following files:

```
NEURO_LAB/
|
|-- main.sce
|-- gui.sci
|-- callbacks.sci
+-- neural_network.sci
```

main.sce

The main entry point of the application. It loads the required Scilab files and starts the GUI.

gui.sci

Contains the graphical user interface including:

- Dataset selection
- Learning-rate input
- Epoch input
- Hidden-neuron input
- Train button
- Reset button
- Network inspection
- Mathematics information
- Results area
- Prediction table
- Graph areas

callbacks.sci

Contains the main interactive functions. It handles:

- Dataset selection
- Training
- Forward propagation
- Loss calculation
- Backpropagation
- Prediction
- Accuracy calculation
- Loss graph
- Neural-network visualization
- Reset
- Network inspection
- Mathematics display

The current callback implementation stores trained weights, activations, predictions, loss history and training parameters in `NEUROLAB_MODEL`.

neural_network.sci

Contains the neural-network-related functionality used by the application.

9. Datasets

The application uses four possible input combinations:

X1	X2
0	0
0	1
1	0
1	1

Three logical problems are supported:

XOR

X1	X2	Output
0	0	0
0	1	1
1	0	1
1	1	0

AND

X1	X2	Output
0	0	0
0	1	0
1	0	0
1	1	1

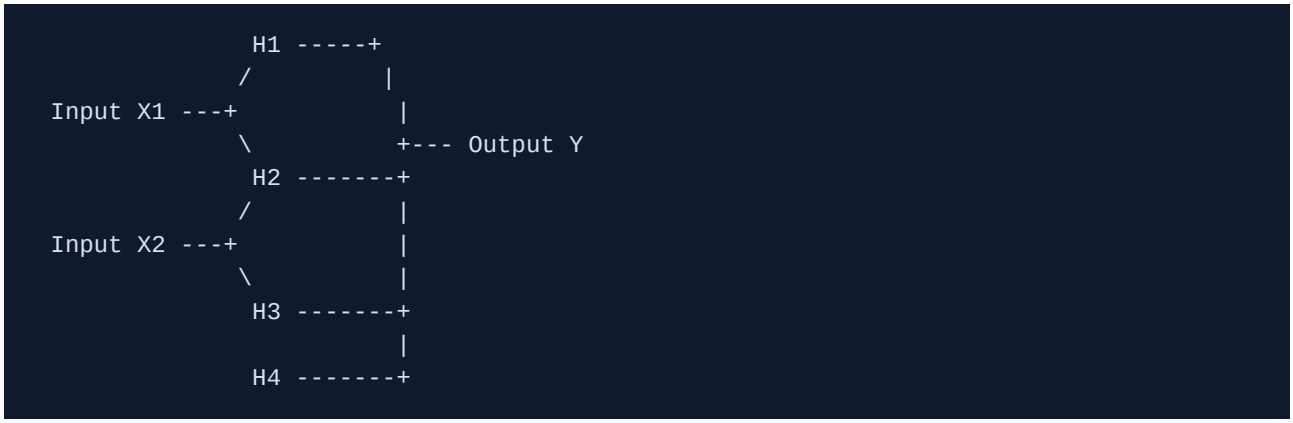
OR

X1	X2	Output
0	0	0
0	1	1
1	0	1
1	1	1

These datasets are directly defined in the application.

10. Neural Network Architecture

The application uses a simple feed-forward neural network:



The input layer contains **2 neurons**, corresponding to X1 and X2.

The hidden layer contains a user-selected number of neurons.

The output layer contains **1 neuron** for binary classification.

The number of hidden neurons can be changed through the GUI. The callback creates the first weight matrix with dimensions `2 x hidden_size` and the second with dimensions `hidden_size x 1`.

11. Neural Network Training

The training process consists of the following steps:

```
Input Data
    |
Weight Initialization
    |
Forward Propagation
    |
Prediction
    |
Loss Calculation
    |
Backpropagation
    |
Gradient Calculation
    |
Weight Update
    |
Repeat for selected epochs
    |
Final Prediction
```

11.1 Weight Initialization

The network starts with randomly initialized weights.

```
W1 = Input -> Hidden weights
W2 = Hidden -> Output weights
```

Bias values are also initialized.

11.2 Forward Propagation

The hidden layer calculates:

```
Z1 = X . W1 + b1
```

Then sigmoid activation is applied:

```
A1 = sigmoid(Z1)
```

The output layer calculates:

```
Z2 = A1 . W2 + b2
```

and produces:

```
A2 = sigmoid(Z2)
```

The implementation follows these calculations directly.

12. Activation Function

The project uses the **Sigmoid activation function**:

```
sigmoid(z) = 1 / (1 + e^-z)
```

The sigmoid output lies between:

```
0  and  1
```

This makes it suitable for binary classification.

13. Loss Function

The application uses **Binary Cross-Entropy Loss**. The loss is calculated as:

```
Loss = -mean[
    y * log(p) +
    (1 - y) * log(1 - p)
]
```

A small epsilon value is used to prevent numerical problems when probabilities become very close to 0 or 1.

14. Backpropagation

After calculating the prediction, the difference between predicted and actual values is calculated:

```
dZ2 = A2 - Y
```

Gradients are then calculated for:

```
W2
b2
W1
b1
```

Finally, the weights are updated using gradient descent:

```
W = W - learning_rate * gradient
```

This process is repeated for the selected number of epochs.

15. GUI Features

Dataset Selection

Users can select:

- XOR
- AND
- OR

Learning Rate

The user can specify the learning rate. Example:

```
0.5
```

A larger learning rate can make learning faster, while an unsuitable learning rate can make training unstable.

Epochs

The number of training iterations can be specified. Example:

```
3000
```

Higher epochs allow the network to perform more weight updates.

Hidden Neurons

The user can select the number of hidden neurons. Example:

```
4
```

The network visualization changes according to the selected hidden-layer size.

16. Training Loss Graph

After training, NEURO LAB displays a graph showing:

```
X-axis -> Epoch
Y-axis -> Loss
```

The graph allows the user to observe how the loss changes during training. Generally:

```
Higher loss
    |
Training
    |
Lower loss
```

A decreasing loss indicates that the network is improving its predictions.

17. Neural Network Visualization

The GUI displays the neural network flow:

```
INPUT -> HIDDEN -> OUTPUT
```

The visualization represents:

- Input neurons
- Hidden neurons
- Output neuron
- Connections between neurons
- Learned weight signs

The application uses:

```
CYAN -> Positive weight
RED  -> Negative weight
```

The network visualization is generated from the actual trained weight matrices rather than being a static diagram.

18. Prediction Table

After training, the application displays:

X1	X2	Actual	Predicted	Probability
0	0	0	0	...

X1	X2	Actual	Predicted	Probability
0	1	1	1	...
1	0	1	1	...
1	1	0	0	...

The probability represents the neural network's output before applying the 0.5 classification threshold.

The application converts probabilities greater than or equal to 0.5 into class 1, otherwise class 0.

19. Accuracy

Accuracy is calculated using:

```
Accuracy = (Correct Predictions / Total Predictions) x 100
```

For example, if all four XOR samples are correctly classified:

```
Accuracy = 4 / 4 x 100
         = 100%
```

20. Network Inspection

The **INSPECT NETWORK** button provides information about the trained model, including:

- Number of input neurons
- Number of hidden neurons
- Number of output neurons
- Weight matrix dimensions
- Sample prediction probability

The model information is stored after training for inspection.

21. Mathematics Feature

The **SHOW MATHEMATICS** button explains the main mathematical operations used by the network:

```
Z1 = X . W1 + b1
A1 = sigmoid(Z1)
A2 = sigmoid(A1 . W2 + b2)
Loss = Binary Cross-Entropy
W = W - learning_rate x gradient
```

This feature helps students connect the visual GUI with the mathematical theory.

22. Reset Feature

The **RESET** button returns the application to its initial state. It resets:

- Dataset
- Learning rate
- Epochs
- Hidden neurons
- Accuracy
- Loss
- Prediction table
- Graphs
- Network visualization
- Stored model

The current implementation explicitly resets the GUI values and clears both visualization axes.

23. Steps to Run the Application

1. Install Scilab 2026.1.0.

2. Place all project files inside one folder (NEURO_LAB/ containing main.sce, gui.sci, callbacks.sci, neural_network.sci).

3. Open Scilab.

4. Set the current working directory to the project folder, e.g. `C:\Users\<YourName>\Desktop\NEURO_LAB`.

5. Open `main.sce`.

6. Run the file — the NeuroLab GUI should open.

7. Select a dataset. For the demonstration use: Dataset = XOR, Learning Rate = 0.5, Epochs = 3000, Hidden Neurons = 4.

8. Click **TRAIN NETWORK**.

9. Observe the accuracy, final loss, prediction table, loss graph, neural network diagram and learning insights.

24. Sample Result

A sample execution can produce results such as:

```
Dataset:        AND
Learning Rate:  0.5
Hidden Neurons: 4
Epochs:         3000

Accuracy:   100%
Final Loss: 0.0021
```

The exact loss and probabilities may vary depending on initialization and training settings.

25. Expected Output

The final GUI should demonstrate:

Left Side

```
Dataset
Learning Rate
Epochs
Hidden Neurons

TRAIN NETWORK
RESET
INSPECT NETWORK
SHOW MATHEMATICS
```

Center

```
NEURAL NETWORK FLOW

Input -> Hidden -> Output
```

```
TRAINING LOSS

Loss
 ^
 |\
 | \
 |  \
 |   \____
 +------------> Epoch
```

Right Side

```
Accuracy
Final Loss
Epochs

PREDICTION TABLE

X1 | X2 | Actual | Predicted | Probability
```

26. Advantages

- Simple and beginner-friendly.
- Interactive GUI.
- No external dataset required.
- Demonstrates actual neural-network training.
- Allows parameter experimentation.
- Provides visual network representation.
- Displays prediction probabilities.
- Displays training loss.
- Helps connect mathematical concepts with visual learning.

27. Limitations

The project is designed primarily for **educational demonstration**. Current limitations include:

- Only small logical datasets are included.
- The network is designed for binary classification.
- Only sigmoid activation is used in the current simplified implementation.
- It is not intended for large real-world datasets.
- Training is performed using a small neural network rather than an optimized deep-learning framework.

28. Future Enhancements

1. Add more activation functions such as ReLU and Tanh.
2. Support custom datasets.
3. Add multiple hidden layers.
4. Add decision-boundary visualization.
5. Add real-time training animation.
6. Add confusion matrix.
7. Add precision, recall and F1-score.
8. Add model export/import.
9. Add adjustable network architecture.
10. Add interactive weight visualization.
11. Add more machine-learning demonstrations.

29. References

Resource	Link
Scilab Official Website	https://www.scilab.org/
Scilab Documentation	https://help.scilab.org/
Scilab GUI Hackathon — Final Submission	https://scilab.in/gui-hackathon/final-submission

30. Conclusion

NEURO LAB provides an interactive way to understand the fundamentals of neural networks. Instead of only presenting equations, the application allows users to modify neural-network parameters and observe the resulting predictions, accuracy, loss and network structure.

The project demonstrates the complete basic learning cycle:

```
DATA
  |
FORWARD PROPAGATION
  |
PREDICTION
  |
LOSS
  |
BACKPROPAGATION
  |
WEIGHT UPDATE
  |
LEARNING
```

Summary

NEURO LAB can serve as a simple educational laboratory for students beginning their journey into Artificial Intelligence and Machine Learning.

Recommended Final Submission Folder

```
NEURO_LAB/
|
|-- Source_Code/
|   |-- main.sce
|   |-- gui.sci
|   |-- callbacks.sci
|   +-- neural_network.sci
|
|-- Screenshots/
|   |-- main_gui.png
|   |-- trained_network.png
|   +-- prediction_results.png
|
|-- Documentation/
|   |-- README.md
|   +-- Problem_Statement.pdf
|
+-- Demo/
    +-- NeuroLab_Demo.mp4
```

Tip for the 2-minute demo video: don't explain every line of code. Show: problem → GUI → choose XOR → change parameters → Train → neural network appears → loss graph → prediction table → accuracy → Inspect/Math button → final result.